```
  concertId,
  rowCount,
  seatsPerRow,
  ticketsToBuyCount,
  venueData,
}) => {
  const rowNumbers = Array.from(Array(rowCount).keys())
  return rowNumbers.map((rowNumber: number) => (
    <Row
      concertId={concertId}
      key={rowNumber}
      rowData={venueData[rowNumber]}
      rowNumber={rowNumber}
      seatsPerRow={seatsPerRow}
      ticketsToBuyCount={ticketsToBuyCount}
    />
  ))
}

export const VenueBody = (props: VenueBodyProps): React.ReactElement => {
  return (
    <table className="table">
      <tbody>{rowItems(props)}</tbody>
    </table>
  )
}

export default VenueBody
```

The `Row` component carries a lot of load here. It maintains a client-side status of each seat, which is based on its ticket status and also the number of tickets the user is looking to buy, so as to prevent buying tickets so close to an already sold ticket that you can't buy an entire block. It also has the click handler invoked when a user clicks on a seat. Here's the new version of the row:

chapter_08/04/app/javascript/components/row.tsx

```
import * as React from "react"
import Seat from "./seat"
import { RowData, TicketData } from "./venue"

interface RowProps {
  concertId: number
```